



Drönarsvärm

Användarmanual Drönarsvärm

2025/12/04

Användarmanual

Adam Mejri, Edvard Wetind, Isac Widendahl
Linus Gustafsson, Oskar Haapaniemi, Oskar Herling, William Olsson

2025/12/04

Version 0.2



Drönarsvärm

Status

Granskad	Isac Widendahl	2025-12-02
Godkänd	Joel Wendin	2015-xx-xx



Projektidentitet

Grupp E-post: dronarsvarm.visionen-users@liuonline.onmicrosoft.com

Hemsida: <http://www.liu.se/grouppage>

Beställare: Joel Wendin, Linköpings universitet
E-post: joel.wendin@liu.se

Kund: Daniel Axehill, Linköpings universitet
E-post: daniel.axehill@liu.se

Expertstöd: Martin Castor, Geistt/Inhumate
E-post: martin.castor@et.mil.se

Expertstöd: Erik Strid, Geistt/Inhumate
E-post: erik.strid@et.mil.se

Handledare: Joel Wikner, Linköpings universitet
E-post: joel.wikner@liu.se

Kursansvarig: Daniel Axehill, Linköpings universitet
E-post: daniel.axehill@liu.se

Projektdeltagare

Namn	Initialer	Ansvar	E-post
Adam Mejri	AM	Projektledare	adame022@student.liu.se
Edvard Wetind	EW	Mjukvaruansvarig	edvwe024@student.liu.se
Isac Widendahl	IW	Dokumentansvarig	isawi003@student.liu.se
Linus Gustafsson	LG	Informationsansvarig	lingu061@student.liu.se
Oskar Haapaniemi	OH	Simuleringsansvarig	oskha100@student.liu.se
Oskar Herling	OHE	Testansvarig	oskhe678@student.liu.se
William Olsson	WO	Designansvarig	wilol978@student.liu.se



INNEHÅLL

1	Installation	1
1.1	Felsökning	1
2	Körning	2
2.1	Kom igång snabbt	2
2.2	Förklaring av användargränssnitt	3
3	Inställningar	5
3.1	PatternLauncher	5
3.2	Drone.prefab	6
3.3	RRT_planner	7
4	Simulering i Visionen	9
	Referenser	10



DOKUMENTHISTORIK

Version	Datum	Utförda förändringar	Utförda av	Granskad
0.2	2015-12-03	Förtydligade kapitel 4 samt fler referenser allmänt	OHE, WO, IW	IW
0.1	2015-11-28	Första utkast	OHE, WO, IW	IW



1 INSTALLATION

Programmet är en utbyggnad på GEISTTs simuleringsmiljö, för att installera den miljön, se GEISTTs manualer [1] [2]. Användaren antas ha en grundläggande förståelse för Unity och vid frågor hänvisas läsaren till att titta igenom grundläggande instruktioner för Unity, till exempel på [Unitys hemsida](#).

När miljön är installerad rekommenderas det att simuleringen körs i några av scenerna som finns i `Packages/com.inhuate.swarms/Tests/Runtime/Demo_Scener`.

Det finns en hel del funktionalitet som är implementerad som inte beskrivs i den här manualen. Många av dessa funktioner är helt enkelt inte genomarbetade nog för att fungera pålitligt för en användare som inte är insatt i miljön. Den här texten är alltså begränsad till att enbart beskriva den huvudsakliga funktionaliteten.

1.1 Felsökning

Simuleringsmiljön är ganska invecklad, därför finns det också mycket som kan gå fel men ett första steg är att läsa på om alla inställningar som beskrivs i kapitel 3 och se till att de har rimliga värden.



2 KÖRNING

2.1 Kom igång snabbt

- Starta Unity, välj en scen och kör den, på så vis startas simuleringen.
- Tryck på två valfria punkter på kartan uppe i vänster hörn, den första är startpunkt och den andra är målpunkt.
- Markera vilken formation drönarna ska anta enligt någon av de fem knapparna längst ner i användargränssnittet.
- För att avsöka ett område, markera [Define Area] och tryck på minst tre valfria punkter i närheten av målpunkten. Välj den sista punkten nära den första punkten för att omsluta ett område.
- Välj avsökningsmetod enligt någon av de tre knapparna längst ner i användargränssnittet.
- Välj antal drönare och tryck på knappen [Launch and Start Mission].
- För att påbörja nytt uppdrag, vänta tills drönarna slutfört uppdraget eller tryck på knappen [Abort Mission].

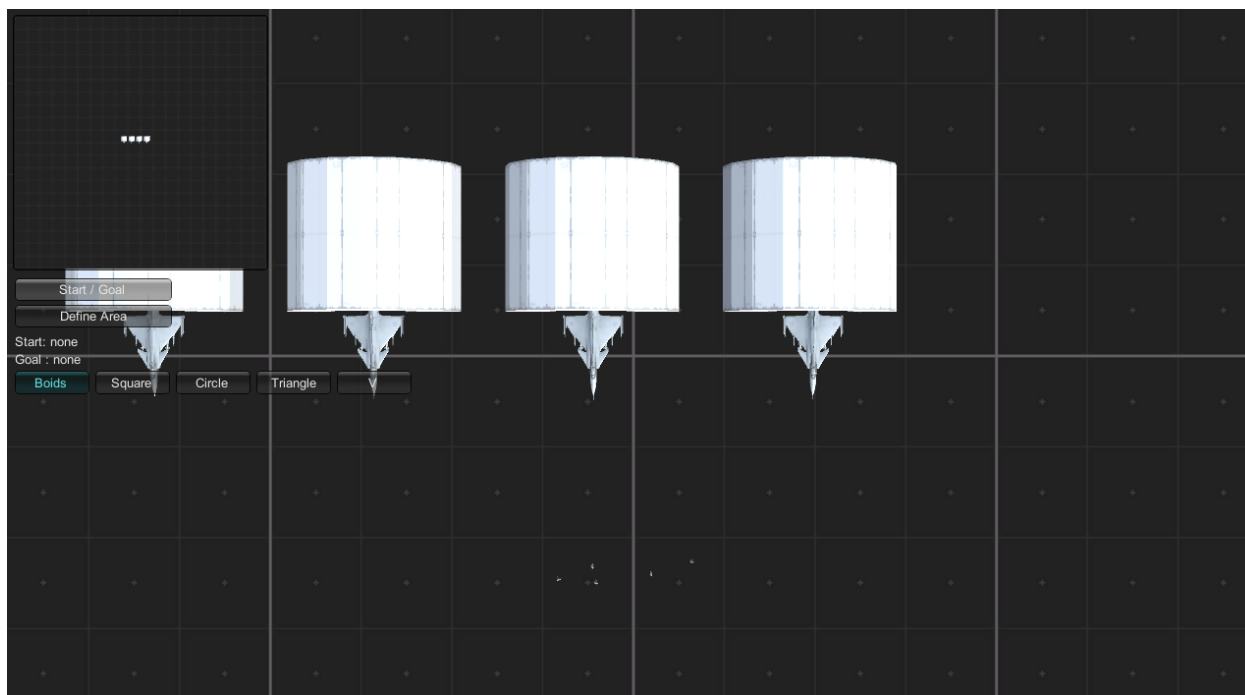
2.2 Förklaring av användargränssnitt

När simuleringen startas visas användargränssnittet uppe i vänster hörn, se figur 1, med alternativ för hur drönarsvärmen initieras enligt följande:

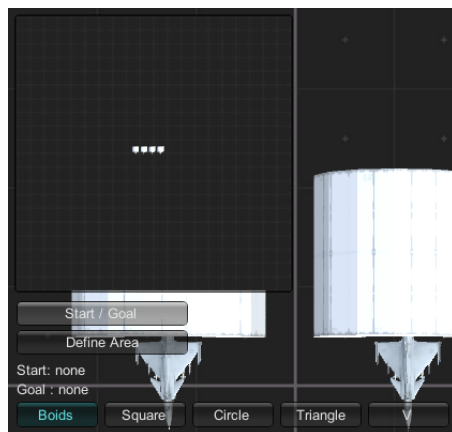
- **Start/Goal:** När denna knapp är markerad kan en väg planeras och formation väljas. Detta sker genom att välja två punkter i kartan placerad i vänster hörn och därefter välja vilken formation svärmen ska färdas i. Formationen väljs genom att markera någon av de fem knapparna [Boids], [Square], [Circle], [Triangle] eller [V], se figur 2a.
- **Define Area:** När denna knapp är markerad kan ett avsökningsområde väljas. Detta sker genom att välja 3 eller fler punkter tills ett område är omslutet. Det går även att ångra punkter med knappen [Undo Last Point] samt välja vilken avsökningsmetod som skall användas. Avsökningsmetod väljs genom att markera en av de tre knapparna [Snake], [Straight Line] eller [Voronoi], se figur 2b.

När en väg har valts visas knappen [Launch and Start Mission] och antalet drönare som ska ingå i svärmen. Denna knapp initialiserar antalet drönare som valts och startar de uppdrag som har valts tidigare. Alltså antingen ett väguppdrag eller både ett väg- och avsöknings-uppdrag, se figur 2c

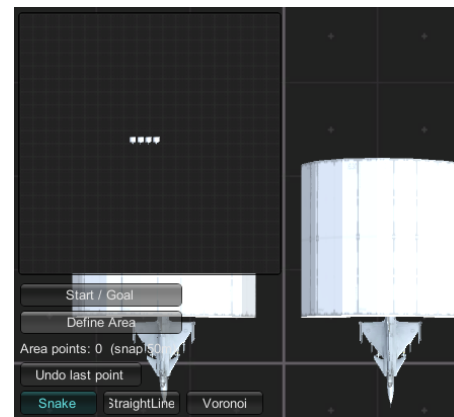
När [Launch and Start Mission] markeras så startar drönarsvärmen sitt uppdrag och knappen [Abort Mission] kommer upp, se figur 2d. Drönarna kommer då fullfölja de uppdrag som valts till de är klara eller [Abort Mission] markeras. Då återgår användargränssnittet till figur 2c och nya uppdrag kan väljas enligt tidigare.



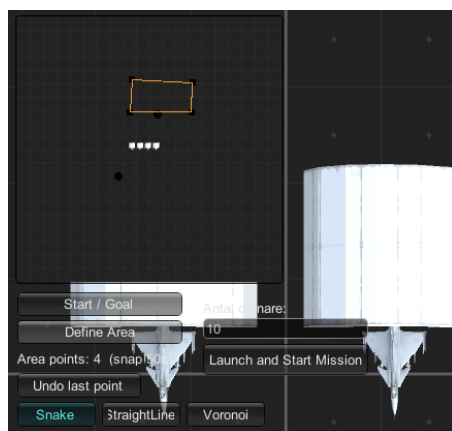
Figur 1: Startskärm simulering



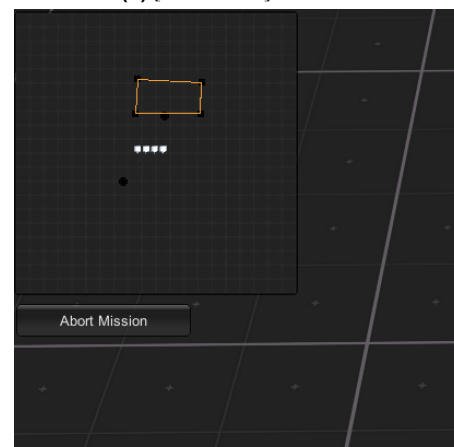
(a) [Start / Goal] markerad



(b) [Defina Area] markerad



(c) Uppdrag valt



(d) Uppdrag påbörjat

Figur 2: Användargränssnitt



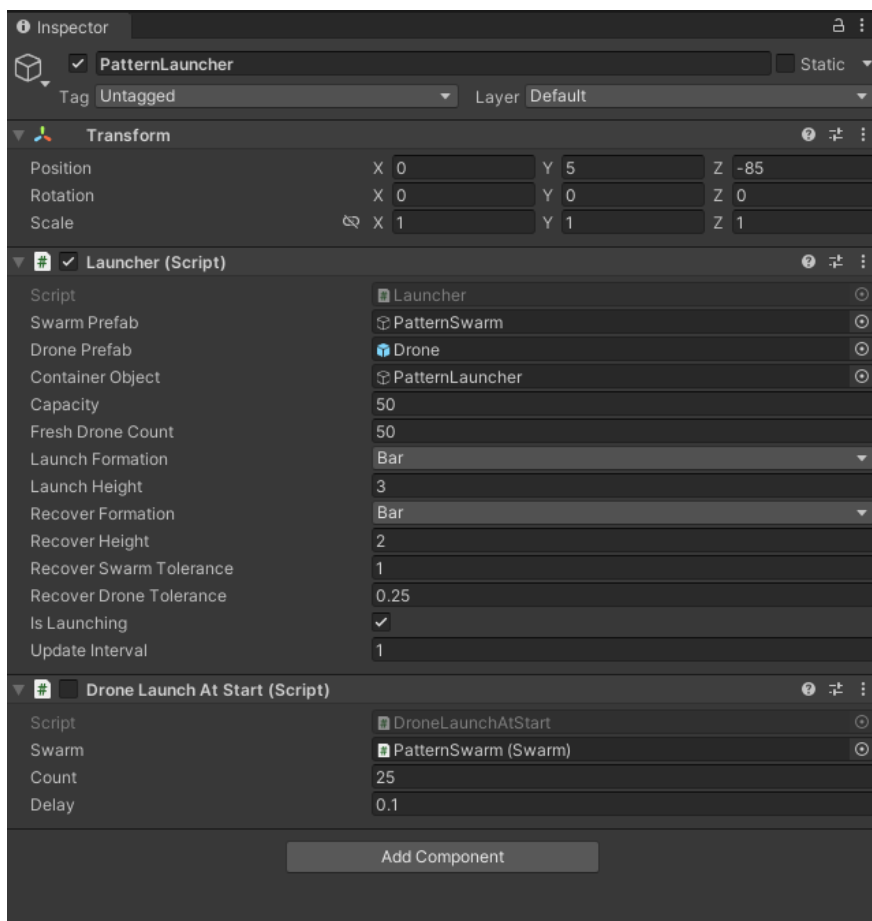
3 INSTÄLLNINGAR

Här beskrivs några olika Inställningar som kan vara intressanta att ändra om systemet inte beter sig som önskat. Många parametrar har ingen påverkan på simuleringen och finns antingen som en rest från tidigare implementationer eller som finns kvar av Unity-tekniska skäl. De viktiga parametrarna beskrivs i det här avsnittet.

3.1 PatternLauncher

Objektet PatternLauncher är ett gameObject i Unity som startar drönarna. En bild på dess komponenter finns i Figur 3. Intressanta inställningar är:

- Capacity & Fresh Drone Count - Hur många drönare som vi har tillgång till. De bör ha samma värde.
- Drone Prefab - Vilken drönarmodell som PatternLauncher startar, bör vara Drone.prefab eller en kopia av Drone.prefab

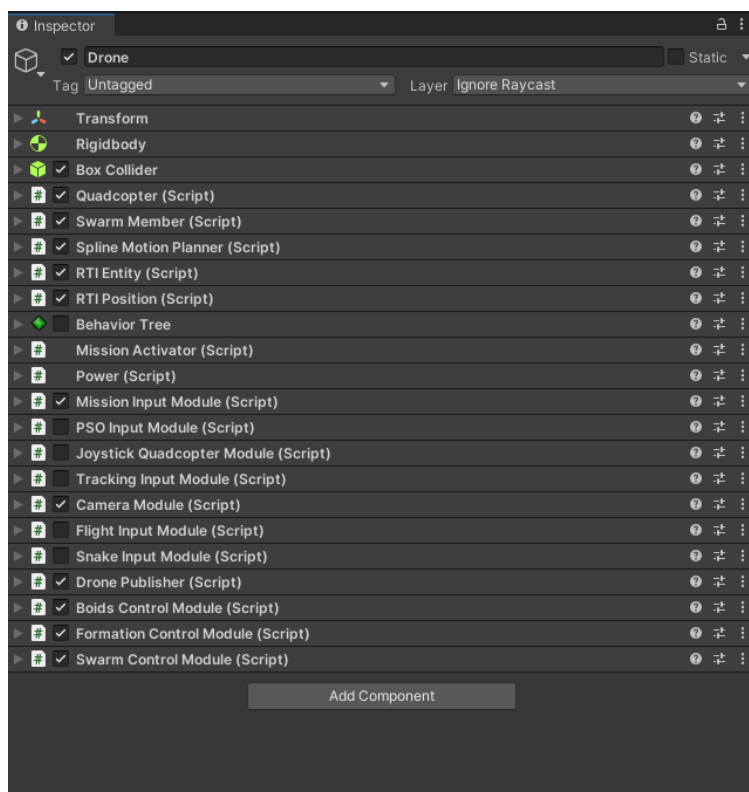


Figur 3: Unityobjektet PatternLauncher med typiska inställningar.

3.2 Drone.prefab

I `Drone.prefab` definieras hur den individuella drönaren fungerar. Alla komponenter som finns i `Drone.prefab` ses i Figur 4. Notera att det går att stänga av komponenter, det kan göra att simuleringen körs snabbare men kan även leda till kryptiska fel om drönarna behöver använda komponenten. I huvudsak finns det användbara inställningar i följande komponenter.

- `BoidsControlModule`
- `FormationControlModule`



Figur 4: Alla komponenter i `Drone.prefab`

I `BoidsControlModule` är följande inställningar intressanta:

- `Perception Radius` - Hur långt en drönare kan se, drönaren får information från alla andra drönare inom `Perception Radius`
- `Seperation Radius` - Hur nära en drönare ska vara en annan drönare för att vilja undvika den, för detaljer se den tekniska rapporten.
- `Boid Weights` - Hur stor vikt varje BOID-kraft får, för detaljer se tekniska rapporten.



- `Collision Radius` - Hur bred drönaren tror att den är när den undviker kollisioner, större värde ger försiktigare drönare.
- `Collision Clearance` - Hur långt fram en drönare tittar för att undvika kollision.
- `Collision Override Distance` - Hur långt från ett hinder drönaren kan vara innan den ignorerar övriga BOID-krafter och fokuserar på att enbart undvika hindret.
- `Goal Reached Radius` - Hur nära drönaren ska vara målet för att anse att den är färdig, i allmänhet rekommenderas ett större värde, fler detaljer finns i tekniska rapporten.

I `FormationControlModule` är följande inställningar intressanta:

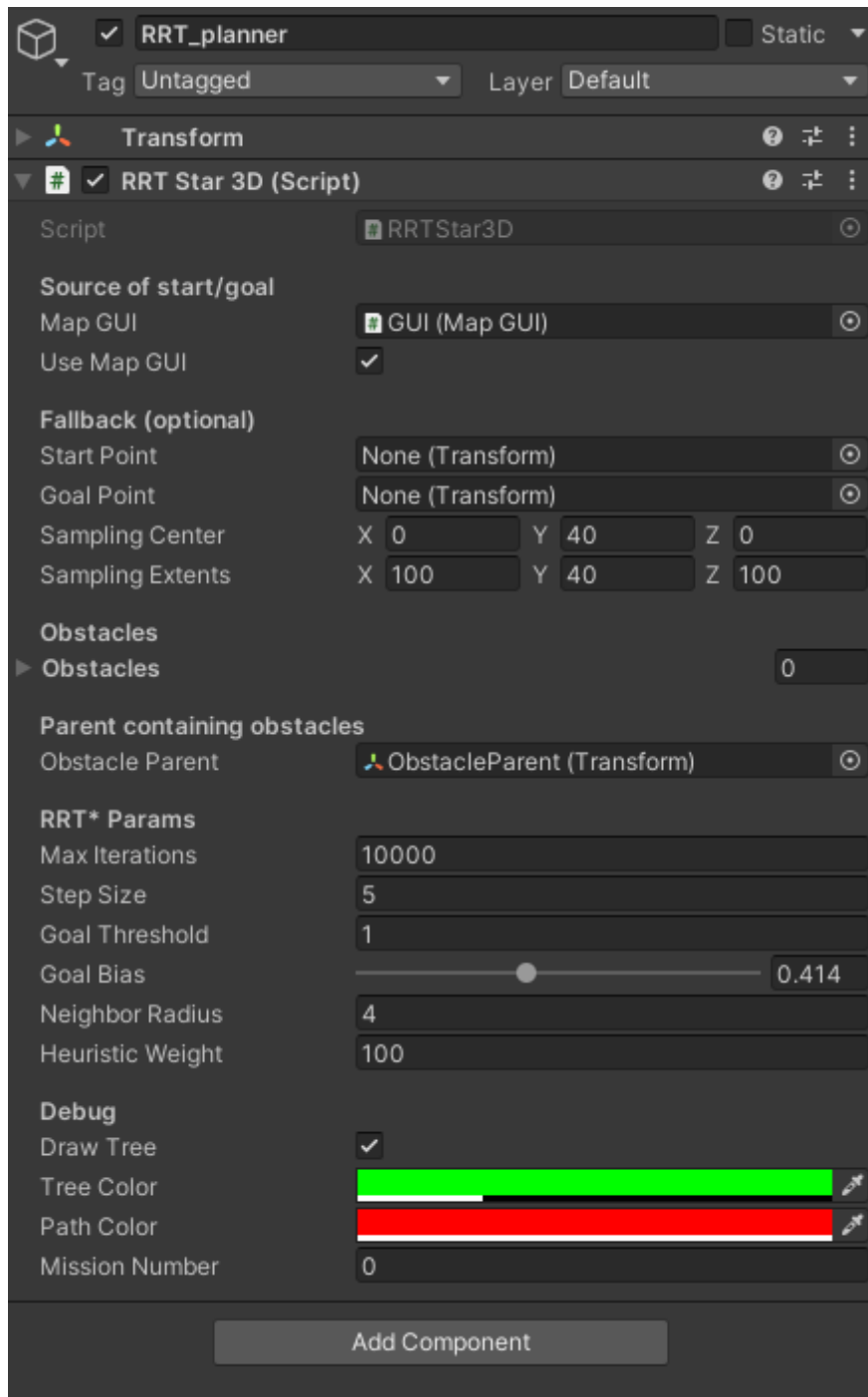
- `Formation Size` - Hur stor formationen kommer vara.
- `Movement Speed` - Hur snabbt formationen rör sig.
- `Perception Radius` - Samma som i `BoidsControlModule`
- `P, I & D` - Inställningar för en PID regulator som styr position.
- `Max Force` - Största styrkraften en drönare kan använda.
- `Obstacle Avoidance Weight` - Hur stor vikt kollisionsundvikningen får jämfört med formationsföljningen
- `Collision Radius & Collision Clearance` - Samma som i `BoidsControlModule`

3.3 RRT_planner

I `RRT_planner` sköts vägplaneringen som sedan skickas vidare till drönaren. För förståelse rekommenderas det att läsa på om RRT* algoritmen innan man ändrar inställningarna, t.ex i den tekniska rapporten. Den ser ut som i Figur 5

Intressanta inställningar är:

- `Max Iterations` - Hur många iterationer av RRT* som körs innan algoritmen bryts och planeraren väljer den billigaste vägen.
- `Step Size` - Hur långa steg RRT* tar i varje iteration
- `Goal Threshold` - Hur nära RRT* vägen måste komma målet för att vägen ska anses färdig.
- `Goal Bias` - Hur stor sannolikheten är att algoritmen väljer en punkt innanför Goal Threshold vid varje iteration.
- `Neighbor radius` - Avgör vilka noder som anses vara grannar.
- `Heuristic Weight` - Den heuristiska vikten är i detta fallet en faktor i kostnadsfunktionen som beror av euklidisk distans från mål.



Figur 5: Exempel på RRT_planner



4 SIMULERING I VISIONEN

Det är möjligt att få en eller flera verkliga drönare att spegla drönarnas positioner i simuleringen. Detta görs då via Visionens Qualisys positioneringssystem och användaren hänvisas därför till deras documentation [3] för konfiguration av Visionen. Det är viktigt att notera att simulering i Visionen endast är implementerat för Crazyflies.

För att kunna köra Crazyflie:n måste hårdvaran vara korrekt laddad, d.v.s uppdaterade drivrutiner. Detta görs manuellt och hur man gör kan hittas på Crazyflie's hemsida [4]. Man behöver också installera paket på sin laptop vilket också hittas på Crazyflie's hemsida [5]. `cfclient` är typiskt bra för att utföra tester eller felsöka drönaren. Se [6] ifall en egen regulator behövs implementeras.

Kommunikationen mellan simuleringen och drönaren sker över RTI som är gränssnittet skapat av Inhumate. Användaren hänvisas därför till Inhumates manual på hur man installerar och konfigurerar RTI [7]. Pythonscriptet `swarms/crazy_fly/cf_rti.py` i projektets kodbas är där kommunikationen från RTI till den verkliga drönaren görs. Scriptet översätter simuleringens koordinatsystem till Visionens koordinatsystem samt innehåller säkerhetskontroller för att inte drönare ska kollidera med väggar eller varandra.

Viktigt: Vid oväntat beteende trycker man på `esc` vilket avbryter de riktiga drönarnas flygning och gör så de landar där de är.

Viktigt: För att ändra antalet drönare som skuggar virtuella drönare, ändras `cf_body_names`, `cf_uris` och `cf_marker_ids`, för fler detaljer se Visionens användarmanual [3].

För att få drönaren att följa simuleringen måste man följa stegen nedan i ordning då det är kritiskt att man t.ex. inte startar drönarens script innan simuleringen.

1. Starta simuleringen och påbörja ett uppdrag. Ett uppdrag måste ha påbörjats annars kan inte scriptet på verkliga drönaren hitta drönarna i simuleringen.
2. Starta python-scriptet `crazy_fly/cf_rti.py`.

`cf_rti.py` börjar med att placera drönarna på en halvmeters höjd på fördefinierade startpositioner i 20 sekunder. Sedan skuggar drönarna de simulerade drönarna i 130 sekunder och avslutar sedan med att landa.



REFERENSER

- [1] Geistt-AB, “liu-swarms,” <https://github.com/geistt-ab/liu-swarms>, accessed: 2025-11-28.
- [2] —, “liu-mip,” <https://github.com/geistt-ab/liu-mip>, accessed: 2025-11-28.
- [3] Visionen, “Documents,” <https://gitlab.liu.se/visionen/documents>, commit: ca5eb630.
- [4] bitcraze, “building-and-flashing,” <https://www.bitcraze.io/documentation/repository/crazyflie-firmware/master/building-and-flashing/build/>, accessed: 2025-11-28.
- [5] —, “cfclients-install,” <https://www.bitcraze.io/documentation/repository/crazyflie-clients-python/master/installation/install/>, accessed: 2025-11-28.
- [6] —, “estimator-or-controller,” <https://www.bitcraze.io/2023/02/adding-an-estimator-or-controller/>, accessed: 2025-11-28.
- [7] “Inhumate systems rti documentation,” <https://docs.inhumatesystems.com/applications/rti/>, accessed: 2025-11-28.